

LAB BOOK

SUBJECT: - CRYPTOGRAPHY
SUBJECT CODE: - 05MC0302

NAME: - NEHA RAJVIR

Enrollment No.: - 92500584015

Faculty Name: - Prof. Vivek Gondaliya

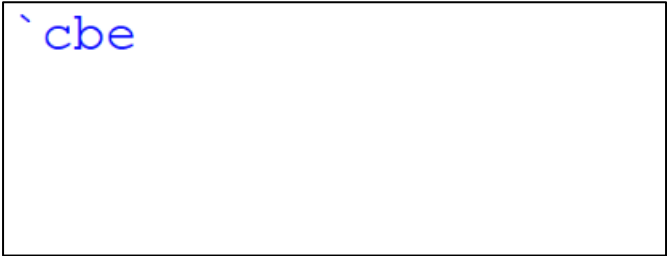
Department: - FOCA (MCA3A)

Q.1 Write a program that contains a string with a value “Hello World”. The program should XOR each character in this string with 0 and displays the result.

Code:

```
string = "abcd"  
result = ""  
  
for char in string:  
    # XOR the character with 1  
    xor_char = chr(ord(char) ^ 1)  
    result += xor_char  
print(result)
```

Output:



cbda

Q.2 Write program to implement Caesar cipher.

Code:

```
def caesar_cipher(message, shift):
    encrypted_message = ""

    for char in message:
        if char.isupper():
            encrypted_char = chr((ord(char) + shift - 65) % 26 + 65)
        elif char.islower():
            encrypted_char = chr((ord(char) + shift - 97) % 26 + 97)
        else:
            encrypted_char = char
        encrypted_message += encrypted_char

    return encrypted_message

message = "Hello, world!"
shift = 3
encrypted_message = caesar_cipher(message, shift)

print("Original message:", message)
print("Shift:", shift)
print("Encrypted message:", encrypted_message)
```

Output:

```
Original message: Hello, world!
Shift: 3
Encrypted message: Khoor, zruog!
```

Q.3 Write a program to implement 2x2 Hill cipher.

Code:

```
import numpy as np

# Function to find modular inverse
def mod_inverse(a, m):
    for i in range(1, m):
        if (a * i) % m == 1:
            return i
    return None # If inverse doesn't exist

# Function to encrypt using 2x2 Hill cipher
def hill_cipher_2x2_encrypt(plaintext, key):
    plaintext = plaintext.upper().replace(" ", "")
    if len(plaintext) % 2 != 0:
        plaintext += 'X' # Padding

    key_matrix = np.array(key).reshape(2, 2)
    ciphertext = ""

    for i in range(0, len(plaintext), 2):
        pair = np.array([[ord(plaintext[i]) - 65], [ord(plaintext[i+1]) - 65]])
        result = np.dot(key_matrix, pair) % 26
        ciphertext += chr(result[0][0] + 65) + chr(result[1][0] + 65)

    return ciphertext

# Function to decrypt using 2x2 Hill cipher
def hill_cipher_2x2_decrypt(ciphertext, key, size):
    key_matrix = np.array(key).reshape(2, 2)
    a, b = key_matrix[0]
    c, d = key_matrix[1]
    det = int((a * d - b * c) % 26)

    inv_det = mod_inverse(det, 26)
    if inv_det is None:
        raise ValueError("Key matrix is not invertible modulo 26.")

    # Compute adjugate matrix and multiply by modular inverse of determinant
    adj = np.array([[d, -b], [-c, a]]) % 26
    inv_key = (inv_det * adj) % 26

    plaintext = ""
    for i in range(0, len(ciphertext), 2):
        pair = np.array([[ord(ciphertext[i]) - 65], [ord(ciphertext[i+1]) - 65]])
        result = np.dot(inv_key, pair) % 26
        plaintext += chr(int(result[0][0]) + 65) + chr(int(result[1][0]) + 65)
```

```
if size % 2 != 0:
    plaintext = plaintext[:size] # Remove padding

return plaintext

# Example usage
plaintext = "HELLO"
size = len(plaintext)
key = [[3, 4], [2, 3]]

ciphertext = hill_cipher_2x2_encrypt(plaintext, key)
decrypted_plaintext = hill_cipher_2x2_decrypt(ciphertext, key, size)

print("Plaintext:", plaintext)
print("Key:", key)
print("Ciphertext:", ciphertext)
print("Decrypted plaintext:", decrypted_plaintext)
```

Output:

```
Plaintext: HELLO
Key: [[3, 4], [2, 3]]
Ciphertext: LAZDET
Decrypted plaintext: HELLO
```

Q.4 Write a Program to implement playfair Cipher.

Code :

```
def playfair_cipher(plaintext, key):
    # Clean and prepare the plaintext
    plaintext = plaintext.upper().replace("J", "I").replace(" ", "")

    # Build the key matrix (5x5)
    key = key.upper().replace("J", "I")
    matrix = []
    for char in key:
        if char not in matrix and char.isalpha():
            matrix.append(char)
    for char in "ABCDEFGHIKLMNOPQRSTUVWXYZ": # 'J' excluded
        if char not in matrix:
            matrix.append(char)
    matrix = [matrix[i*5:(i+1)*5] for i in range(5)]

    # Helper to find position of letter in matrix
    def find_position(letter):
        for i in range(5):
            for j in range(5):
                if matrix[i][j] == letter:
                    return i, j

    # Prepare plaintext digraphs
    i = 0
    pairs = []
    while i < len(plaintext):
        a = plaintext[i]
        b = plaintext[i+1] if i+1 < len(plaintext) else 'X'
        if a == b:
            pairs.append(a + 'X')
            i += 1
        else:
            pairs.append(a + b)
            i += 2
    if len(pairs[-1]) == 1:
        pairs[-1] += 'X'

    # Encrypt the pairs
    ciphertext = ""
    for pair in pairs:
        a, b = pair[0], pair[1]
        row1, col1 = find_position(a)
        row2, col2 = find_position(b)
```

```
    if row1 == row2:
        ciphertext += matrix[row1][(col1 + 1) % 5]
        ciphertext += matrix[row2][(col2 + 1) % 5]
    elif col1 == col2:
        ciphertext += matrix[(row1 + 1) % 5][col1]
        ciphertext += matrix[(row2 + 1) % 5][col2]
    else:
        ciphertext += matrix[row1][col2]
        ciphertext += matrix[row2][col1]

return ciphertext

# Example usage
plaintext = "HELLO WORLD"
key = "example"
ciphertext = playfair_cipher(plaintext, key)

print("Plaintext:", plaintext)
print("Key:", key)
print("Ciphertext:", ciphertext)
```

Output:

```
Plaintext: HELLO WORLD
Key: example
Ciphertext: GXBEGUUROCBM
```

Q.5 Write program to implement Rail Fence Cipher.

Code:

```
def rail_fence_cipher(plaintext, rails):
    fence = [[] for i in range(rails)]
    rail = 0
    direction = 1
    for letter in plaintext:
        fence[rail].append(letter)
        rail += direction
        if rail == rails-1 or rail == 0:
            direction = -direction

    ciphertext = ""
    for rail in fence:
        for letter in rail:
            ciphertext += letter

    return ciphertext

plaintext = "HELLO WORLD"
rails = 3
ciphertext = rail_fence_cipher(plaintext, rails)

print("Plaintext:", plaintext)
print("Rails:", rails)
print("Ciphertext:", ciphertext)
```

Output:

```
Plaintext: HELLO WORLD
Rails: 3
Ciphertext: HOREL OLLWD
```

Q.6 Write program to implement DES.

Code:

```
#pip install pycryptodomex or pip install pycryptodome or pip install cryptodome

from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad

def encrypt_ecb(data, key):
    cipher = DES.new(key, DES.MODE_ECB)
    ciphertext = cipher.encrypt(pad(data, 8))
    return ciphertext

def decrypt_ecb(ciphertext, key):
    cipher = DES.new(key, DES.MODE_ECB)
    plaintext = unpad(cipher.decrypt(ciphertext), 8)
    return plaintext

def encrypt_cbc(data, key, iv):
    cipher = DES.new(key, DES.MODE_CBC, iv)
    ciphertext = cipher.encrypt(pad(data, 8))
    return ciphertext

def decrypt_cbc(ciphertext, key, iv):
    cipher = DES.new(key, DES.MODE_CBC, iv)
    plaintext = unpad(cipher.decrypt(ciphertext), 8)
    return plaintext

def encrypt_cfb(data, key, iv):
    cipher = DES.new(key, DES.MODE_CFB, iv)
    ciphertext = cipher.encrypt(data)
    return ciphertext

def decrypt_cfb(ciphertext, key, iv):
    cipher = DES.new(key, DES.MODE_CFB, iv)
    plaintext = cipher.decrypt(ciphertext)
    return plaintext

def main():
    mode = input("Select mode (ECB, CBC, CFB): ").upper()
    data = input("Enter plain text: ").encode()
    key = input("Enter key (8 bytes): ").encode()
    #iv = get_random_bytes(8) # Initialization vector for CBC and CFB modes
    iv = input("Enter initialization vector for CBC and CFB modes (8 bytes): ").encode()

    if len(key) != 8:
        print("Key must be 8 bytes long.")
        return
```

```

if mode == "ECB":
    ciphertext = encrypt_ecb(data, key)
    decrypted_text = decrypt_ecb(ciphertext, key)
elif mode == "CBC":
    ciphertext = encrypt_cbc(data, key, iv)
    decrypted_text = decrypt_cbc(ciphertext, key, iv)
elif mode == "CFB":
    ciphertext = encrypt_cfb(data, key, iv)
    decrypted_text = decrypt_cfb(ciphertext, key, iv)
else:
    print("Invalid mode selected.")
    return

print("Ciphertext:", ciphertext.hex())
print("Decrypted text:", decrypted_text.decode())

if __name__ == "__main__":
    main()

```

Output:

```

Select mode (ECB, CBC, CFB): CBC
Enter plain text: Hello World
Enter key (8 bytes): 12345678
Enter initialization vector for CBC and CFB modes (8 bytes): 87654321
Ciphertext: 4a0073cbf9fc0b2cd98da08b741fb4e9
Decrypted text: Hello World
|

```

Q.7 WAP Encryption and decryption using 3DES Algorithm.

Code:

```
#pip install pycryptodomex or pip install pycryptodome or pip install cryptodome
from Crypto.Cipher import DES3
from Crypto.Util.Padding import pad, unpad
from Crypto.Random import get_random_bytes

def encrypt_ecb(plaintext, key):
    cipher = DES3.new(key, DES3.MODE_ECB)
    ciphertext = cipher.encrypt(pad(plaintext, 8))
    return ciphertext

def decrypt_ecb(ciphertext, key):
    cipher = DES3.new(key, DES3.MODE_ECB)
    decrypted = cipher.decrypt(ciphertext)
    return unpad(decrypted, 8)

def encrypt_cbc(plaintext, key, iv):
    cipher = DES3.new(key, DES3.MODE_CBC, iv)
    ciphertext = cipher.encrypt(pad(plaintext, 8))
    return ciphertext

def decrypt_cbc(ciphertext, key, iv):
    cipher = DES3.new(key, DES3.MODE_CBC, iv)
    decrypted = cipher.decrypt(ciphertext)
    return unpad(decrypted, 8)

def encrypt_cfb(plaintext, key, iv):
    cipher = DES3.new(key, DES3.MODE_CFB, iv)
    ciphertext = cipher.encrypt(plaintext)
    return ciphertext

def decrypt_cfb(ciphertext, key, iv):
    cipher = DES3.new(key, DES3.MODE_CFB, iv)
    decrypted = cipher.decrypt(ciphertext)
    return decrypted

def main():
    plaintext = input("Enter the plaintext: ").encode()
    key = input("Enter the 3DES key (16 or 24 bytes): ").encode()
    iv = get_random_bytes(8) # Initialization Vector for CBC and CFB

    print("Select encryption mode:")
    print("1. ECB")
    print("2. CBC")
    print("3. CFB")
    mode_choice = int(input("Enter mode choice: "))
```

```

if mode_choice == 1:
    encrypted = encrypt_ecb(plaintext, key)
    decrypted = decrypt_ecb(encrypted, key)
    mode_name = "ECB"
elif mode_choice == 2:
    encrypted = encrypt_cbc(plaintext, key, iv)
    decrypted = decrypt_cbc(encrypted, key, iv)
    mode_name = "CBC"
elif mode_choice == 3:
    encrypted = encrypt_cfb(plaintext, key, iv)
    decrypted = decrypt_cfb(encrypted, key, iv)
    mode_name = "CFB"
else:
    print("Invalid mode choice")
    return

print("\n" + mode_name + ":")
print("Plaintext:", plaintext.decode())
print("Encrypted:", encrypted)
print("Decrypted:", decrypted.decode())

if __name__ == "__main__":
    main()

```

Output:

```

===== RESTART: C:\Users\hp\Desktop\crypto practical
Enter the plaintext: Hello World
Enter the 3DES key (16 or 24 bytes): 1234567890123456
Select encryption mode:
1. ECB
2. CBC
3. CFB
Enter mode choice: 2

CBC:
Plaintext: Hello World
Encrypted: b'2\xd6\x93j\xfa\xd3rRA\xb1\x08e\xde|\x11\xeb'
Decrypted: Hello World

```

Q.8 WAP Encryption and decryption using AES Algorithm.

Code:

```
#pip install pycryptodomex or pip install pycryptodome or pip install cryptodome
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from Crypto.Random import get_random_bytes

def encrypt_ecb(plaintext, key):
    cipher = AES.new(key, AES.MODE_ECB)
    ciphertext = cipher.encrypt(pad(plaintext, AES.block_size))
    return ciphertext

def decrypt_ecb(ciphertext, key):
    cipher = AES.new(key, AES.MODE_ECB)
    decrypted = cipher.decrypt(ciphertext)
    return unpad(decrypted, AES.block_size)

def encrypt_cbc(plaintext, key, iv):
    cipher = AES.new(key, AES.MODE_CBC, iv)
    ciphertext = cipher.encrypt(pad(plaintext, AES.block_size))
    return ciphertext

def decrypt_cbc(ciphertext, key, iv):
    cipher = AES.new(key, AES.MODE_CBC, iv)
    decrypted = cipher.decrypt(ciphertext)
    return unpad(decrypted, AES.block_size)

def encrypt_cfb(plaintext, key, iv):
    cipher = AES.new(key, AES.MODE_CFB, iv)
    ciphertext = cipher.encrypt(plaintext)
    return ciphertext

def decrypt_cfb(ciphertext, key, iv):
    cipher = AES.new(key, AES.MODE_CFB, iv)
    decrypted = cipher.decrypt(ciphertext)
    return decrypted

def main():
    plaintext = input("Enter the plaintext: ").encode()
    key = input("Enter the AES key (16, 24, or 32 bytes): ").encode()
    iv = get_random_bytes(AES.block_size) # Initialization Vector for CBC and CFB

    print("Select encryption mode:")
    print("1. ECB")
    print("2. CBC")
    print("3. CFB")
    mode_choice = int(input("Enter mode choice: "))
```

```

if mode_choice == 1:
    encrypted = encrypt_ecb(plaintext, key)
    decrypted = decrypt_ecb(encrypted, key)
    mode_name = "ECB"
elif mode_choice == 2:
    encrypted = encrypt_cbc(plaintext, key, iv)
    decrypted = decrypt_cbc(encrypted, key, iv)
    mode_name = "CBC"
elif mode_choice == 3:
    encrypted = encrypt_cfb(plaintext, key, iv)
    decrypted = decrypt_cfb(encrypted, key, iv)
    mode_name = "CFB"
else:
    print("Invalid mode choice")
    return

print("\n" + mode_name + ":")
print("Plaintext:", plaintext.decode())
print("Encrypted:", encrypted.hex())
print("Decrypted:", decrypted.decode())

if __name__ == "__main__":
    main()

```

Output:

```

Enter the plaintext: Hello World
Enter the AES key (16, 24, or 32 bytes): 1234567890123456
Select encryption mode:
1. ECB
2. CBC
3. CFB
Enter mode choice: 2

CBC:
Plaintext: Hello World
Encrypted: 57f687fa19f6e3843c3c2ca2d277dd2b
Decrypted: Hello World

```

Q.9 Implement a stand-alone sender receiver program in which Sender encrypts a given text message entered from keyboard using RSA public key of Receiver and sends it to Receiver. Receiver Decrypts the cipher-text using his own RSA Private key and display the decrypted contents on screen. Use Appropriate package/Library for RSA.

9.1 Generate Key:

Code:

```
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization

def generate_keys():
    # Generate private key
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048
    )

    # Save private key
    with open("receiver_private.pem", "wb") as f:
        f.write(private_key.private_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PrivateFormat.TraditionalOpenSSL,
            encryption_algorithm=serialization.NoEncryption()
        ))

    # Generate and save public key
    public_key = private_key.public_key()
    with open("receiver_public.pem", "wb") as f:
        f.write(public_key.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo
        ))

    print("✅ RSA Key pair generated successfully.")

if __name__ == "__main__":
    generate_keys()
```

Output:

```
===== RESTART: C:\Users\hp\Desktop\crypt
✅ RSA Key pair generated successfully.
```

9.2 RSA Receiver:

Code:

```
# pip install cryptography
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric.padding import OAEP, MGF1
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa

def receiver():
    try:
        # Load the receiver's private key
        with open('receiver_private.pem', 'rb') as f:
            receiver_private_key = serialization.load_pem_private_key(
                f.read(),
                password=None
            )
    except FileNotFoundError:
        print("Error: 'receiver_private.pem' file not found. Please generate the private key first.")
        return

    try:
        # Load the encrypted message
        with open('encrypted_message.bin', 'rb') as f:
            ciphertext = f.read()
    except FileNotFoundError:
        print("Error: 'encrypted_message.bin' file not found. Please make sure the sender has encrypted and saved the message.")
        return

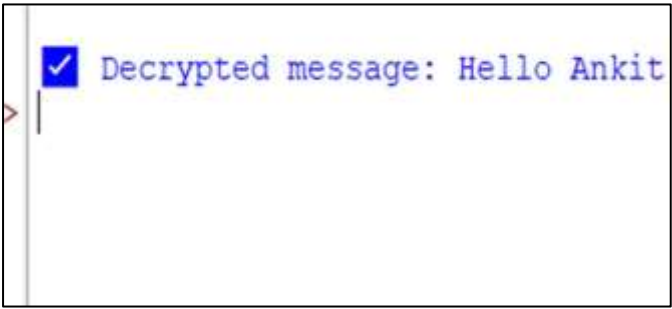
    try:
        # Decrypt the message using RSA private key with OAEP padding
        plaintext = receiver_private_key.decrypt(
            ciphertext,
            OAEP(
                mgf=MGF1(algorithm=hashes.SHA256()),
                algorithm=hashes.SHA256(),
                label=None
            )
        )
```

)

```
print("\n✅ Decrypted message:", plaintext.decode())  
except Exception as e:  
    print("❌ Decryption failed:", str(e))
```

```
if __name__ == "__main__":  
    receiver()
```

Output:



```
> |  
✅ Decrypted message: Hello Ankit
```

9.3 RAS Sender:

Code:

```
# pip install cryptography
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import hashes, serialization

def sender():
    try:
        # Load receiver's public key
        with open('receiver_public.pem', 'rb') as f:
            receiver_public_key = serialization.load_pem_public_key(f.read())
    except FileNotFoundError:
        print("Error: 'receiver_public.pem' file not found. Run key generation first.")
        return

    # Take input message from user
    message = input("Enter the message to encrypt and send: ").encode()

    try:
        # Encrypt message using RSA public key and OAEP padding
        ciphertext = receiver_public_key.encrypt(
            message,
            padding.OAEP(
                mgf=padding.MGF1(algorithm=hashes.SHA256()),
                algorithm=hashes.SHA256(),
                label=None
            )
        )

        # Save encrypted message to file
        with open('encrypted_message.bin', 'wb') as f:
            f.write(ciphertext)
```

```
print("✅ Message encrypted and saved as 'encrypted_message.bin'.")  
except Exception as e:  
    print("❌ Encryption failed:", str(e))
```

```
if __name__ == "__main__":  
    sender()
```

Output:

```
===== RESTART: C:\Users\hp\Desktop\crypto practical\9.RSA  
Enter the message to encrypt and send: Hello Ankit  
✅ Message encrypted and saved as 'encrypted_message.bin'.  
|
```

Q.10 Implement a stand-alone sender receiver program in which Sender encrypts a given text message entered from keyboard in a byte-by-byte manner using RC4 Stream Cipher and transmits it to receiver. Receiver Decrypts the ciphertext displays the decrypted contents on screen. Use Appropriate package/Library for RC4.

10.1 RC4_Receiver:

Code:

```
#pip install receiver
from Crypto.Cipher import ARC4

def receiver():
    # Load the encrypted message from a file or receive it
    with open('encrypted_message.bin', 'rb') as f:
        ciphertext = f.read()

    # Enter the same shared secret key (password) used for encryption
    key = input("Enter the secret key (password) used for encryption: ").encode()

    # Create an RC4 cipher with the provided key
    cipher = ARC4.new(key)

    # Decrypt the ciphertext byte-by-byte
    plaintext = cipher.decrypt(ciphertext)

    print("Decrypted message:", plaintext.decode())

if __name__ == "__main__":
    receiver()
```

Output:

```
===== RESTART: C:\Users\hp\Desktop\crypto practical\9.RSA _
Enter the secret key (password) used for encryption: 12345
Decrypted message: I am a Student
|
```

10.2 RC4_Sender:

Code:

```
#pip install cryptography
from Crypto.Cipher import ARC4

def sender():
    # Get the plaintext message from the user
    plaintext = input("Enter the message: ").encode()

    # Enter a shared secret key (password) for encryption
    key = input("Enter a secret key (password): ").encode()

    # Create an RC4 cipher with the provided key
    cipher = ARC4.new(key)

    # Encrypt the message byte-by-byte
    ciphertext = cipher.encrypt(plaintext)

    # Save the encrypted message to a file or transmit it to the receiver
    with open('encrypted_message.bin', 'wb') as f:
        f.write(ciphertext)

    print("Message encrypted and saved as 'encrypted_message.bin'")

if __name__ == "__main__":
    sender()
```

Output:

```
***** RESTART: C:\Users\hp\Desktop\crypto practical
Enter the message: I am a Student
Enter a secret key (password): 12345
Message encrypted and saved as 'encrypted_message.bin'
|
```

Q.11 Write program implement _md5.

Code:

```
import hashlib
import sys

str= input("enter the value ")

str=bytes(str,'utf-8')
result= hashlib.md5(str);
print("the byte equivalent of hash is :", end="")
print(result.digest())

print("\r")
print("the size of output :",end="")
print(sys.getsizeof(result.digest()))

str= input("enter the value ")

str=bytes(str,'utf-8')
result= hashlib.md5(str);
print("the byte equivalent of hash is :", end="")
print(result.digest())

print("\r")
print("the size of output :",end="")
print(sys.getsizeof(result.digest()))
```

Output:

```
enter the value  hello
the byte equivalent of hash is :b']A@*\xbck*v\xb9q\x9d\x91\x10\x17\xc5\x92'

the size of output :49
enter the value
```

Q.12 Write program implement _sha512.

Code:

```
import hashlib

str="abhaydodiya"

result=hashlib.sha512(str.encode())

print(result.hexdigest())
print("\r")
result=hashlib.sha1(str.encode())

print(result.hexdigest())
```

Output:

```
dd446e3b1c1ff03d7b5665b5ace0d9406a4351c179f38e994d13e5d2fd982a19837eb6c99ede572a
7ebae362cfbf97d06a0cfffbd095bd04383e69612171ddb

c05976dbf17d72e62d0521b002faa60f570c6903
```

Q.13 Write program implement _File.

Code:

```
import hashlib

pass1=input("enter the password ")
text_file=open("hello","wb")
pass1=bytes(pass1,"utf-8")
result=hashlib.md5(pass1)
text_file.write(result.digest())
print(result.digest())
text_file.close()
print(" Password hash stored ")
text_file=open("hello","rb")
newpass=input("enter the password again ")
content=text_file.read()
#print(content)
#digest=hashlib.md5("hello".encode("utf-8")).digest()
#print(digest)
#np1=hashlib.sha1(newpass.encode())
#np1=result.hexdigest()
np1=bytes(newpass,"utf-8")
np1=hashlib.md5(np1)
np1=np1.digest()
print(np1)

if content==np1:
    print("login succesfully ")
else:
    print("login denied ")
```

Output:

```
enter the password 12345
b'\x82|\xcb\x0e\xea\x8aplL4\xalh\x91\xf8N{'
 Password hash stored
enter the password again 12345
b'\x82|\xcb\x0e\xea\x8aplL4\xalh\x91\xf8N{'
login succesfully
```

Q.14 Write program implement Diffie-Hellman Key Exchange.

Code:

```
# Function to calculate (base^exp) % mod
def power(base, exp, mod):
    result = 1
    base = base % mod
    while exp > 0:
        if exp % 2 == 1:
            result = (result * base) % mod
        exp = exp // 2
        base = (base * base) % mod
    return result

# Function to perform Diffie-Hellman key exchange
def diffie_hellman(p, g, private_key):
    public_key = power(g, private_key, p)
    return public_key

# Example usage
# Choose large prime numbers for p and g, and private keys for Alice and Bob
p = 23 # Prime number
g = 5  # Primitive root modulo p

# Private keys for Alice and Bob
private_key_alice = 6
private_key_bob = 15

# Calculate public keys for Alice and Bob
public_key_alice = diffie_hellman(p, g, private_key_alice)
public_key_bob = diffie_hellman(p, g, private_key_bob)

# Shared secret key calculation
shared_secret_alice = power(public_key_bob, private_key_alice, p)
shared_secret_bob = power(public_key_alice, private_key_bob, p)

# Output
print("Public key for Alice:", public_key_alice)
print("Public key for Bob:", public_key_bob)
print("Shared Secret Key for Alice:", shared_secret_alice)
print("Shared Secret Key for Bob:", shared_secret_bob)
```

Output:

```
Public key for Alice: 8
Public key for Bob: 19
Shared Secret Key for Alice: 2
Shared Secret Key for Bob: 2
|
```

Q.15 Implement steganography for hiding any kind of data using an image file.

Code:

```
# pip install pillow
from PIL import Image

def encode_image(image_path, data_to_hide):
    image = Image.open(image_path).convert("RGB") # ensure RGB

    # Text -> UTF-8 bytes -> bits, then add 0x00 terminator
    binary_data = ''.join(format(b, '08b') for b in data_to_hide.encode('utf-8')) + '00000000'

    capacity = image.width * image.height * 3 # 3 bits/pixel (R,G,B)
    if len(binary_data) > capacity:
        raise Exception(f'Data too large: {len(binary_data)} bits > capacity {capacity} bits')

    data_index = 0
    for y in range(image.height):
        for x in range(image.width):
            r, g, b = image.getpixel((x, y))
            if data_index < len(binary_data):
                r = (r & ~1) | int(binary_data[data_index]); data_index += 1
            if data_index < len(binary_data):
                g = (g & ~1) | int(binary_data[data_index]); data_index += 1
            if data_index < len(binary_data):
                b = (b & ~1) | int(binary_data[data_index]); data_index += 1
            image.putpixel((x, y), (r, g, b))
            if data_index >= len(binary_data):
                break
        if data_index >= len(binary_data):
            break

    image.save("encoded_image.png")
    print("Data encoded successfully -> encoded_image.png")

def decode_image(encoded_image_path):
    encoded_image = Image.open(encoded_image_path).convert("RGB")
    bit_buffer = ""
    out = bytearray()

    for y in range(encoded_image.height):
        for x in range(encoded_image.width):
            r, g, b = encoded_image.getpixel((x, y))
            for comp in (r, g, b):
```

```
bit_buffer += str(comp & 1)
if len(bit_buffer) >= 8:
    byte_bits = bit_buffer[:8]
    bit_buffer = bit_buffer[8:]
    if byte_bits == '00000000': # terminator
        return out.decode('utf-8', errors='replace')
    out.append(int(byte_bits, 2))
return out.decode('utf-8', errors='replace')
```

Example

```
data_to_hide = "This is a hidden message!"
encode_image("original_image.png", data_to_hide)
decoded_data = decode_image("encoded_image.png")
print("Decoded data:", decoded_data)
```

Output:

```
Data encoded successfully -> encoded_image.png
Decoded data: This is a hidden message!
```

